

Jass (Jaime's home ASsistant Script)

Jaime Antonio Daniel Filho¹

¹Curso de Ciência da Computação –
Universidade Federal de Santa Maria (UFSM)

jafilho@inf.ufsm.br

1. Introdução

O Home Assistant consolidou-se como uma das principais plataformas de automação residencial, permitindo integrar e controlar dispositivos de diferentes fabricantes em um único ambiente. Entretanto, a criação de automações é baseada em arquivos YAML (*YAML Ain't Markup Language*), cuja estrutura pode se tornar extensa e complexa mesmo para tarefas simples. Uma automação básica, como acender uma luz ao abrir uma porta, exige o conhecimento de diversos elementos da sintaxe, como gatilhos, condições, ações, serviços e parâmetros específicos de cada dispositivo. Além disso, muitos erros só são percebidos durante a execução da automação, seja por um nome de serviço incorreto, um parâmetro incompatível ou um tipo de valor inválido, dificultando a identificação da causa do problema e tornando o processo de desenvolvimento menos acessível para usuários sem experiência técnica.

Com o objetivo de reduzir essa complexidade, este trabalho apresenta o Jass, uma *Domain-Specific Language* (DSL) capaz de transpilar descrições de alto nível para o formato YAML utilizado pelo Home Assistant. A proposta é oferecer uma sintaxe mais simples e intuitiva, abstraindo detalhes estruturais da plataforma e permitindo que o usuário descreva apenas o comportamento desejado da automação. A linguagem também incorpora mecanismos de validação léxica, sintática e semântica, possibilitando que erros, como construções inválidas ou operações incompatíveis com determinados dispositivos.

A Listagem 1 e a Listagem 2 ilustram, respectivamente, a automação completa em Jass e o YAML equivalente que o compilador gera. Note como `turn_on(light.varanda, ...)` se expande para o serviço `light.turn_on` com a estrutura `target/data`, e como `delay(5m)` vira a string `"00:05:00"` no formato exigido pela plataforma.

```
automation "Luz da varanda" {
  when (binary_sensor.porta changes to "on")
  do {
    turn_on(light.varanda, brightness=180)
    delay(5m)
    turn_off(light.varanda)
  }
}
```

Listagem 1. Uma automação básica em Jass.

```
- id: "1780962966"
  alias: "Luz da varanda"
  mode: "single"
  triggers:
    - trigger: state
      entity_id: binary_sensor.porta
      to: "on"
  actions:
    - action: light.turn_on
      target:
        entity_id: light.varanda
      data:
        brightness: 180
    - delay: "00:05:00"
    - action: light.turn_off
      target:
        entity_id: light.varanda
```

Listagem 2. A saída YAML da automação anterior.

O restante do relatório está organizado da seguinte forma: a Seção 2 apresenta as instruções para execução; a Seção 3 descreve a arquitetura; a Seção 4 define a linguagem de forma conceitual; a Seção 5 a formaliza a gramática livre de contexto; as Seções 6 a 8 detalham as fases de análise; a Seção 9 trata da geração de código; a Seção 10 reúne exemplos completos e a Seção 11 conclui, sintetizando os resultados.

2. Instruções para execução

O projeto usa o gerenciador uv, que pode ser instalado em Linux, macOS e Windows. A partir do diretório do trabalho, instale as dependências e o comando Jass (Listagem 3):

```
uv sync
```

Listagem 3. Comando para configurar o projeto.

Para transpilar um arquivo .jass para YAML (Listagem 4):

```
uv run jass caminho/para/automacao.jass
```

Listagem 4. Comando para transpilar um arquivo .jass para YAML.

Para gravar o resultado em arquivo, ao invés de mostrar no terminal, com `-o` (Listagem 5):

```
uv run jass caminho/para/automacao.jass -o automacao.yaml
```

Listagem 5. Comando para gravar o resultado em arquivo.

Para inspecionar uma fase intermediária do compilador, com `--emit` (Listagem 6), especificando `tokens`, `parse`, `ast`, `check` ou `yaml`:

```
# imprime a AST tipada
uv run jass caminho/para/automacao.jass --emit ast

# roda a análise semântica
uv run jass caminho/para/automacao.jass --emit check
```

Listagem 6. Comando para inspecionar uma fase intermediária do compilador.

Em caso de erro, os diagnósticos são impressos no `stderr` e o programa termina com código não nulo.

3. Visão geral da arquitetura

O Jass é organizado como um compilador em *pipeline*, ou seja, em uma sequência de fases em que cada uma consome o artefato produzido pela anterior e gera o artefato da seguinte. Essa estrutura mantém as responsabilidades separadas e torna o sistema fácil de testar e inspecionar fase a fase. A Tabela 1 associa cada fase ao módulo que a implementa e ao artefato que ela produz.

Fase	Módulo
Análise léxica	<code>parser.py</code> (Lark)
Análise sintática	<code>parser.py</code> (Lark, LALR)
Transformação	<code>transform.py</code> , <code>ast.py</code> , <code>values.py</code>
Análise semântica	<code>analyzer.py</code> , <code>registry.py</code>
Geração de código	<code>codegen.py</code>
Orquestração / diagnósticos	<code>cli.py</code> , <code>errors.py</code>

Tabela 1. Fases do compilador e módulos correspondentes.

Os artefatos intermediários são, em ordem: o texto-fonte; a sequência de tokens, unidades léxicas como palavras-chave, identificadores e literais; a árvore de derivação (*parse tree*), que reflete a estrutura gramatical da entrada; a AST (*Abstract Syntax Tree*), uma representação tipada e enxuta, na qual cada nó é um objeto Python com tipo próprio e posição de origem; o modelo de saída, uma árvore de dicionários e listas Python já na forma esperada pelo Home Assistant; e, por fim, o YAML serializado. A análise semântica é uma fase à parte: percorre a AST para preencher a tabela de símbolos e validar a entrada, produzindo a decisão de que o programa está correto ou a lista de erros.

As duas primeiras fases são construídas sobre o Lark, uma biblioteca Python para construção de analisadores. O Lark recebe uma gramática em notação EBNF e oferece, a partir dela, o *lexer* (analisador léxico) e o *parser* (analisador sintático), além de utilitários para o tratamento de erros. O Jass o emprega na configuração LALR(1) com *lexer* contextual, aproveitando:

- a propagação automática de linha e coluna para cada nó, base dos diagnósticos localizados;
- as exceções de erro distintas para caractere inválido e *token* inesperado, que sustentam a separação entre erro léxico e sintático;
- o *parser* interativo, que permite alimentar *tokens* manualmente e, assim, implementar a recuperação de erros em modo pânico.

A gramática reconhece apenas a forma das construções, isto é, a ordem das seções, o formato de um gatilho, a estrutura de uma chamada, enquanto regras que dependem de tipos, domínios e nomes ficam a cargo do analisador semântico, que tem mais contexto para produzir mensagens precisas.

4. Definição da Linguagem

O projeto da linguagem foi orientado por dois princípios fundamentais. O primeiro é fornecer uma sintaxe declarativa e uniforme, estruturada em seções com responsabilidades bem definidas. O segundo é adotar um vocabulário reduzido e composto por nomes amigáveis, como `turn_on`, `delay` e `say`, cujo significado é conhecido e validado pelo compilador. Esta seção apresenta, em nível conceitual, as principais construções oferecidas pela linguagem.

4.1. Automação

A unidade básica de um programa é a automação, introduzida pela palavra-chave `automation` seguida de um título e de um corpo entre chaves. Um arquivo contém zero ou mais automações e, opcionalmente, definições de nomes. O corpo é composto por seções em ordem fixa: metadados opcionais (`id`, `description`, `mode`, `max`), a seção `when` (gatilhos), a seção `if` (condições) e a seção `do` (ações). A Listagem 7 apresenta o esqueleto de uma automação com as quatro seções. A ordem metadados, `when`, `if` e `do` é imposta pela gramática.

```
automation "Aquecedor da suíte" {  
    mode = "single" // metadado  
    when (switch.toalheiro changes to "on") // gatilho  
    if (between @15:00:00 @23:00:00) // condição  
    do { turn_on(switch.toalheiro) } // ações  
}
```

Listagem 7. Esqueleto de uma automação.

4.2. Gatilhos

A seção `when` lista um ou mais gatilhos, os eventos que disparam a automação, separados pela palavra `or`. Cada gatilho pode receber um identificador com as `"id"`, referenciado depois nas condições. Há quatro famílias, exemplificadas na Listagem 8:

- **de estado:** ENTIDADE changes [from V] [to V] [for DURAÇÃO] reage à mudança de estado de uma entidade, opcionalmente filtrando origem/destino e exigindo permanência por um tempo;
- **numérico:** ENTIDADE (< | >) NÚMERO [for DURAÇÃO] reage ao cruzamento de um limiar;
- **de horário:** @HH:MM:SS dispara em um horário do dia;
- **solar:** (sunset | sunrise) [(+ | -) DURAÇÃO] dispara no nascer ou pôr do sol, com deslocamento opcional.

```
when (
  binary_sensor.sala changes to "on" for 5m as "movimento" or
  sensor.temperatura > 30 as "calor" or
  @07:00:00 as "manha" or
  sunset - 30m as "anoitecer"
)
```

Listagem 8. Exemplos de gatilhos.

4.3. Condições

A seção `if` é uma expressão booleana que precisa ser verdadeira para a automação prosseguir. As condições abrangem:

- **estado:** ENTIDADE == "v" ou !=;
- **numérica:** ENTIDADE < N ou > N;
- **horário:** after @T, before @T, between @T @T, on [dias];
- **solar:** after sunset, before sunrise - 10m, between sunset sunrise com deslocamento opcional;
- **gatilho:** triggeredby "id".

Há suporte também para os operadores lógicos `not`, `and` e `or`, com precedência idêntica à linguagem Python (`not` > `and` > `or`) e parênteses para agrupar. A Listagem 9 exemplifica algumas condições.

```
if (
  light.sala == "on" and
  sensor.temperatura < 28 and
  between @06:00:00 @23:00:00 and
  not triggeredby "calor"
)
```

Listagem 9. Exemplos de condições.

A forma `between A B`, diferentemente de escrever `after A and before B` como dois átomos, gera uma única condição com ambos os limites, o que o Home Assistant interpreta como um intervalo que pode atravessar dias distintos.

4.4. Ações

A seção `do` é uma sequência de comandos, executados em ordem. As formas são chamadas de ação com `nome(arg, ...)`, esperas com `delay(DURAÇÃO)`, condicionais com `if`, `elif` e `else`, e blocos que continuam em caso de erro com `safe`. Na Listagem 10, é possível observar algumas ações.

```
do {
  turn_on(lampada, color=quente, transition=2s)
  delay(5m)
  if (triggeredby "manha") {
    turn_off(switch.luz)
  } else {
    toggle(switch.luz)
  }

  safe {
    say(tts.google, message="Aviso")
  }
}
```

Listagem 10. Exemplos de ações.

As chamadas a serviços dependem diretamente do domínio do alvo. Isso significa que a mesma ação sobre dispositivos diferentes apresenta capacidades distintas. Por exemplo, `turn_on` sobre uma luz aceita `color`, enquanto que sobre um interruptor, não.

Por fim, como não é possível definir um vocabulário de ações e invocações diretas que cubra todos os casos, o Jass permite invocar diretamente serviços com `call("dominio.servico", ...)`, sendo a única ação isenta de validação de argumentos, como demonstra a Listagem 11.

```
call("vacuum.send_command", target=vacuum.roomba, data={"command":
↪ "start"})
```

Listagem 11. Invocação direta de serviços.

4.5. Valores

Os valores escritos em argumentos, metadados e nomes pertencem a um conjunto fixo de tipos, resumido na Tabela 2. Tipos como duração, horário, referência de entidade e cor têm sintaxe própria.

Tipo	Exemplo
string	"on"
número	30, 0.09
booleano	True, False
duração	5m, 1h30m, 45s
horário	@07:00:00
referência de entidade	light.sala
lista	["off", "on"]
dicionário	{"command": "start"}
cor	rgb(255, 180, 80)

Tabela 2. Tipos de valor da linguagem.

4.6. Nomes e variáveis

Para evitar repetição, o Jass permite vincular nomes a valores com `def NOME = VALOR`. Esses nomes são associações de tempo de compilação, existindo apenas durante a compilação e sendo substituídos pelo valor correspondente na saída. Cada uso é verificado por tipo exatamente como se o valor estivesse escrito no lugar.

Há dois escopos: o de arquivo (global, visível a qualquer automação declarada abaixo) e o de bloco (local ao `{ }` em que o nome é declarado, com sombreamento, isto é, um nome local oculta o de mesmo nome em escopo externo. A Listagem 12 ilustra definições globais (`lampada`, `quente`) e local (`reforco`).

```
def lampada = light.sala
def quente  = rgb(255, 180, 80)

automation "Luz aconchegante" {
  when (lampada changes to "on")
  do {
    turn_on(lampada, color=quente)
    def reforco = 100
    turn_on(lampada, brightness=reforco)
  }
}
```

Listagem 12. Exemplos de definições.

5. Gramática Livre de Contexto (GLC)

A sintaxe do Jass é definida no arquivo `src/jass/grammar.lark`, escrito na variante de EBNF (*Extended Backus–Naur Form*) adotada pelo Lark. Essa variante utiliza as seguintes notações para expressar a gramática:

- as regras (não-terminais) são escritas em minúsculas e definidas com `:`, enquanto os terminais são escritos em maiúsculas;
- `|` separa alternativas;

- * indica repetição (zero ou mais), + (uma ou mais) e ? um elemento opcional (zero ou uma ocorrência);
- (...) agrupa elementos;
- um terminal é definido por um literal entre aspas ("when") ou por uma expressão regular entre barras (/.../);
- %ignore descarta um terminal do fluxo (espaços em branco e comentários) e // inicia um comentário no arquivo.

Além desses símbolos, existem duas diretivas que afetam apenas o formato da árvore sintática construída: o prefixo ? em uma regra a embute, removendo o seu nó da árvore quando ele possui um único filho, o que enxuga a árvore resultante; e -> nome atribui um rótulo à alternativa, dando um nome estável ao nó por ela gerado. A Listagem 13 apresenta parcialmente a arquivo com a gramática, que pode ser visto na íntegra no repositório.

```

start: (def_binding | automation)*

// Alias definitions
def_binding: DEF NAME ASSIGN value

// Automation definitions
automation: AUTOMATION STRING? LBRACE metadata* when_block? condition_block? do_block
↳ RBRACE
metadata: NAME ASSIGN value

// Triggers
when_block: WHEN LPAR trigger (OR trigger)* RPAR
trigger: trigger_expr (AS STRING)?
?trigger_expr: state_trigger
               | numeric_trigger
               | time_trigger
               | sun_trigger
state_trigger: entity CHANGES (FROM value)? (TO value)? (FOR DURATION)?
numeric_trigger: entity (LT | GT) threshold (FOR DURATION)?
time_trigger: TIME
sun_trigger: sun_event

// Conditions
condition_block: IF LPAR bool_expr RPAR

...

// Values
TIME: /@{d}{2}:{d}{2}:{d}{2}/
DURATION: /\d+h(\d+m)?(\d+s)?|\d+m(\d+s)?|\d+s/
NUMBER: /-?\d+\.\d+|-?\d+/
ENTITY: /[a-z][a-z_]*\.[a-z0-9_]+/
NAME: /[a-zA-Z_][a-zA-Z0-9_]*/
STRING: /"(\.|\.[^"]\")*/

// Comments
COMMENT: /\s\/\s/[^\n]*/
%ignore COMMENT
%ignore /\s\t\f\r\n\]/

```

Listagem 13. Conteúdo parcial da gramática com os não-terminais e os terminais.

6. Análise léxica

A análise léxica converte o texto-fonte em uma sequência de *tokens*. No Jass, essa fase é inteiramente delegada ao Lark, sendo as definições de terminais declaradas na própria gramática, das quais o Lark deriva o *lexer*. Comentários de linha (`// ...`) e espaços em branco são descartados por diretivas `%ignore`, não chegando ao *parser*.

Quando o *lexer* encontra um caractere que não inicia nenhum terminal, o Lark lança a exceção `UnexpectedCharacters`, que o Jass captura e converte em um erro da categoria léxica, preservando a linha, a coluna e o caractere ofensor. A Listagem 14 mostra o diagnóstico real para uma entrada que contém um `$` inválido.

```
// Caractere que não inicia nenhum token ('$').
automation "Caractere inválido" {
  when (binary_sensor.porta changes to "on")
  do {
    turn_on(light.sala$led)
  }
}
```

```
samples/errors/00-lexical.jass:5:27: lexical error: unexpected
↳ character '$'
    turn_on(light.sala$led)
                        ^
```

Listagem 14. Demonstração de erro léxico.

7. Análise sintática

A análise sintática verifica se a sequência de *tokens* forma um programa válido segundo a gramática e produz a árvore de derivação. O Jass usa o *parser* LALR(1) do Lark, construído a partir da gramática. A análise opera em conjunto com o *lexer* contextual, que usa o estado do *parser* para escolher, quando há ambiguidade léxica, apenas entre os terminais válidos naquele ponto.

Quando o *parser* recebe um *token* que nenhuma entrada da tabela aceita, o Lark sinaliza um *token* inesperado, que o Jass converte em um erro da categoria sintática, com posição e a lista de *tokens* esperados naquele ponto. Em vez de parar no primeiro erro, o Jass aplica recuperação em modo pânico, isto é, ao detectar a falha, registra o erro, descarta *tokens* até alcançar um ponto de sincronização e retoma a análise a partir dele. Os pontos de sincronização são as fronteiras de topo do programa, as palavras-chave `automation` e `def`, que marcam o início de uma nova unidade independente. Assim, um erro em uma automação não contamina as seguintes, e uma única execução coleta todos os erros. Alguns exemplos de erros sintáticos são apresentados na Listagem 15.

```

// Falta a seção 'do' (obrigatória).
automation "Bloco do ausente" {
    when (binary_sensor.porta changes to "on")
        if (alarm_control_panel.alarmo == "disarmed")
    }

// 'or' pendente: espera-se um gatilho após o último 'or'.
automation "Or pendente" {
    when (
        binary_sensor.sala changes to "on" as "sala" or
        binary_sensor.corredor changes to "on" as "corredor" or
    )
    do {
        turn_on(light.corredor)
    }
}

// Seções fora de ordem: 'if' antes de 'when'.
automation "Seções fora de ordem" {
    if (alarm_control_panel.alarmo == "disarmed")
    when (@08:00:00 as "manha")
    do { }
}

```

```

03-syntactic.jass:5:1: syntactic error: unexpected '>'; expected 'do'
    }
    ^
03-syntactic.jass:12:5: syntactic error: unexpected ')'; expected
→ 'sunrise', 'sunset', a name, a time literal, ...
    )
    ^
03-syntactic.jass:20:5: syntactic error: unexpected 'if'; expected
→ 'when', a name
    if (alarm_control_panel.alarmo == "disarmed")
    ^^

```

Listagem 15. Demonstração de erros sintáticos.

8. Análise semântica

A análise semântica é a fase em que o Jass concentra a maior parte de suas verificações, percorrendo a AST de cada automação e coletando todos os erros. A análise é baseada em três pilares, tratados a seguir: a tabela de símbolos, a verificação de tipos e a consistência ação e domínio. A Listagem 16 mostra alguns exemplos de erros semânticos.

```

automation "Declaração" {
    when(light.sala changes)

    do {
        turn_on(luz)
    }
}

automation "Tipo" {
    when(light.sala changes)

    do {
        turn_on(light.sala, color=2s)
    }
}

automation "Ação e domínio" {
    when(light.sala changes)

    do {
        turn_on([light.sala, timer.sala])
    }
}

```

```

05-semantic.jass:5:17: semantic error: undeclared name 'luz'
    turn_on(luz)
           ^^^

05-semantic.jass:13:35: semantic error: argument 'color' expects a
→ color, got a duration
    turn_on(light.sala, color=2s)
                        ^^

05-semantic.jass:21:17: semantic error: all entities in a target list
→ must share one domain (found: light, timer)
    turn_on([light.sala, timer.sala])
            ^^^^^^^^^^^^^^^^^^^^^^^

```

Listagem 16. Demonstração de erros semânticos.

8.1. Tabela de símbolos e escopos

A tabela de símbolos registra os nomes vinculados por `def`, associando a cada um o tipo inferido do valor e o próprio valor. Os tipos formam um conjunto fixo de categorias, como string, número, booleano, duração, horário, entidade (que carrega também o domínio), lista, dicionário.

Os escopos são organizados em uma pilha que reflete a estrutura do programa: o escopo de arquivo (global), o de cada automação e o de cada bloco. A resolução de um nome percorre a pilha do escopo atual para fora. Isso significa que uma definição local de mesmo nome “sombreia” a externa. Nomes não resolvidos são erros. As referências dos gatilhos declaradas com as `"id"` são coletadas por automação e formam o conjunto contra o qual cada `triggeredby "id"` é validado.

8.2. Verificação de tipos

Cada valor usado em um argumento, gatilho ou condição tem seu tipo inferido e comparado ao tipo esperado naquele ponto. Quando o valor é uma referência a uma definição, ela é resolvida ao seu tipo antes de qualquer verificação.

8.3. Consistência ação e domínio

O significado de uma ação depende do domínio do alvo, isto é, do tipo do dispositivo indicado pelo primeiro parâmetro. O processo de análise ocorre de forma sequencial, começando pela determinação do domínio do alvo, com a exigência de que todas as entidades de um alvo em lista compartilhem o mesmo domínio. Em seguida, verifica-se se a ação está devidamente definida para ele. Por fim, validam-se os argumentos, garantindo que cada um seja aceito pela ação naquele domínio, apresente o tipo correto e respeite a obrigatoriedade, a ausência de duplicatas e a ligação posicional pela ordem. A única exceção a esse procedimento de análise é `call(...)`, que é isenta de validação.

9. Geração de código

A geração de código traduz a AST validada para a estrutura de saída do Home Assistant. A AST é primeiro convertida em um modelo de saída, um conjunto de dicionários e listas Python, já no formato da plataforma. Um `id` é gerado, caso não tenha sido especificado. Em seguida, o modelo é serializado em YAML, com a biblioteca `ruamel.yaml`.

10. Exemplos

Os exemplos completos estão contidos no diretório `samples`, com suas transpilações correspondentes. No entanto, para fins de demonstração no relatório, a Listagem 17 mostra a fonte de uma automação real, e a Listagem 18 exibe o YAML completo gerado para ela.

```
def lampada    = light.sala
def amarelo    = rgb(255, 150, 60)
def tempo      = 5m

automation "Luz Sala" {
  description = "Ligar a luz quando ha movimento ou ao anoitecer"
  mode = "restart"

  when (
    binary_sensor.sala changes to "on" as "movimento" or
    sunset - 15m as "anoitecer"
  )

  if (
    between sunset sunrise and
    alarm_control_panel.casa == "disarmed"
  )

  do {
    turn_on(lampada, color=amarelo, transition=2s)

    if (triggeredby "movimento") {
      delay(tempo)
      turn_off(lampada)
    } else {
      say(tts.sala, player=media_player.sala, message="Bom
        ↪ anoitecer")
    }
  }
}
```

Listagem 17. Automação de exemplo em Jass.

```
- id: "1780965949"
  alias: "Luz Sala"
  description: "Ligar a luz quando ha movimento ou ao anoitecer"
  mode: "restart"
  triggers:
    - trigger: state
      entity_id: binary_sensor.sala
      to: "on"
      id: "movimento"
    - trigger: sun
      event: sunset
      offset: "-00:15:00"
      id: "anoitecer"
  conditions:
    - condition: sun
      after: sunset
      before: sunrise
    - condition: state
      entity_id: alarm_control_panel.casa
      state: "disarmed"
  actions:
    - action: light.turn_on
      target:
        entity_id: light.sala
      data:
        rgb_color:
          - 255
          - 150
          - 60
        transition: 2
    - if:
      - condition: trigger
        id: "movimento"
      then:
        - delay: "00:05:00"
        - action: light.turn_off
          target:
            entity_id: light.sala
      else:
        - action: tts.speak
          target:
            entity_id: tts.sala
          data:
            media_player_entity_id: media_player.sala
            message: "Bom anoitecer"
```

Listagem 18. Saída completa em YAML para a Listagem 17.

11. Conclusão

O Jass cumpre os objetivos propostos, oferecendo uma sintaxe declarativa e legível que esconde a complexidade do YAML de automações do Home Assistant e permite a detecção de erros em tempo de compilação. As falhas léxicas, sintáticas e semânticas são apresentadas com categoria e localização, coletando vários problemas em uma única execução.

Todos os exemplos são transpilados para YAML válido e carregável.